

Lab Report: Lab 3 - Layered Architecture Implementation (CRUD)

Contents

1. Cover Page	1
2. Abstract/Summary	1
3. Lab Specific Section: Layered Architecture Implementation	1
4. Conclusion & Reflection.....	3

1. Cover Page

- **Title:** Implementing Layered Architecture and CRUD Operations for OmniDash
 - **Course Info:** Software Architecture
 - **Student Details:** Nguyễn Văn Trường, 23010371
 - **Date:** April 30, 2026
-

2. Abstract/Summary

The OmniDash project aims to develop a robust workspace automation and task management platform. This lab focused on the practical implementation of the system's core features utilizing a Layered Architecture. While the theoretical model suggests strict folder separations, OmniDash adapts this into a modern, industry-standard monorepo structure separating the frontend client, backend API, and browser extension. We successfully set up the project structure with distinct logical modules representing the Presentation, Business Logic, and Persistence layers. By utilizing Node.js and the Prisma ORM, we established the Create, Read, Update, and Delete (CRUD) operations for the core entities. The primary objective achieved in this lab was enforcing a strict, unidirectional flow of control and dependency from the HTTP entry points down to the database level.

3. Lab Specific Section: Layered Architecture Implementation

This section documents the deliverables for Lab 3, demonstrating how the theoretical concepts of Layered Architecture are mapped to the OmniDash project's physical directory structure and logical execution flow.

3.1 Project Setup and Layer Mapping

To enforce the separation of concerns, the project directory was structured into a monorepo containing distinct execution environments. Instead of abstract theoretical folders, the OmniDash architecture implements the three foundational layers across its backend configuration.

```

+---backend
|   \---prisma
|       \---migrations
|       \---src
|           +---config
|           +---controllers
|           +---middlewares
|           +---routes
|           \---utils
+---Document
+---extension
\---frontend
    \---app
        +---admin
        |   +---system
        |   \---users
        +---components
        +---dashboard
        |   +---calendar
        |   +---notes
        |   +---project
        |       \---[id]
        |   +---projects
        |   +---reminders
        |   +---settings
        |   \---workspaces
        +---forgot-password
        +---login
        +---register
        +---reset-password
        |   \---[token]
        \---store

```

- **Presentation Layer (Client & API Routing):** Physically represented by the frontend directory (Next.js UI) and the backend/src/routes module. This layer is strictly responsible for receiving HTTP requests, capturing user inputs, and returning formatted JSON responses.
- **Business Logic Layer (Controllers & Validation):** Physically represented by the backend/src/controllers and backend/src/middlewares directories. This layer acts as the brain of the operation, enforcing business rules and validating Data Transfer Objects (DTOs) before interacting with the database.
- **Persistence Layer (Data Access):** Physically represented by the backend/prisma directory. Utilizing the Prisma Object-Relational Mapper (ORM), this layer encapsulates all complex SQL query logic and provides an abstract interface for the upper layers.

3.2 Persistence Layer Implementation (Prisma ORM)

In accordance with architectural best practices, the Persistence Layer must interact directly with the data store and remain completely isolated from HTTP request contexts. In the OmniDash system, this is achieved using Prisma ORM.

Rather than manually writing repository classes or simulating a database with in-memory data structures, Prisma automatically generates a type-safe database client based on the defined schema. The backend/prisma/migrations directory tracks all schema changes over time. When the

Business Logic layer requires data retrieval or manipulation, it invokes the Prisma client's built-in methods (such as create, findMany, update, and delete). This ensures that the SQL syntax and database engine details are entirely abstracted away from the core application logic.

3.3 Business Logic Layer Implementation

The Business Logic layer orchestrates the core rules of OmniDash and dictates how data flows between the user and the database. It requires the Persistence layer to access data, strictly enforcing the downward dependency flow.

In the OmniDash structure, this logic is encapsulated within the functions residing in backend/src/controllers and supported by backend/src/middlewares. For instance, when a user attempts to create a new Task or Workspace, the business logic validates the input payload. It checks for required fields, verifies data formats, and enforces domain rules (e.g., ensuring a task title is not empty or verifying that a user has the authorization to modify a specific workspace). If the business rules are violated, this layer immediately throws an error, preventing invalid data from ever reaching the Persistence Layer.

3.4 Presentation Layer Implementation

The Presentation Layer is strictly responsible for handling incoming HTTP communications, delegating the complex processing to the Business Logic Layer, and formatting the outgoing responses. In the backend, the backend/src/routes directory defines the RESTful endpoints (e.g., POST /api/tasks, GET /api/workspaces). These routes contain absolutely no business logic or database queries. Instead, they act as pure traffic directors, mapping a specific URL and HTTP method to the corresponding Controller function. Once the Controller (Business Logic) finishes its execution, the Presentation Layer packages the resulting data or error messages into a standardized JSON format with the appropriate HTTP status codes (e.g., 201 Created, 400 Bad Request, 404 Not Found) to be consumed by the Next.js frontend application.

4. Conclusion & Reflection

Lab 3 successfully transitioned the OmniDash system design into a structured, highly organized codebase. By adapting the theoretical Layered Architecture to a modern Next.js/Node.js stack, we achieved a strict separation of concerns. The Routing layer acts solely as an interface, the Controllers enforce strict OmniDash business rules, and Prisma ORM manages data isolation perfectly.

This architectural discipline ensures that if OmniDash later transitions to a different database engine, only the Prisma schema requires modification, leaving the business logic and API routing entirely unaffected. This structural integrity is a critical stepping stone, preparing the system for the Microservices decomposition and communication strategies that will be explored in subsequent labs.